Configuración de permisos por roles

Entrega individual de Enrique Zavala. Continúa la API financiera existente con Python, Strawberry, GraphQL, autenticación JWT y MySQL. Incluye la configuración de cuatro puestos y la demostración de las cuatro partes del documento de actividad.

Verificación	Resultado real
Regresión y permisos en SQLite	57 pruebas aprobadas
Permisos e integridad en MySQL	15 pruebas aprobadas
Operaciones ejecutadas en GraphiQL	28 respuestas verificadas
Reinicio de la API	4 roles y 10 permisos; sin duplicados
Servidor MySQL	8.4.11
Matriz final	26 parejas activas y 2 parejas inactivas conservadas

Ana: ADMINISTRADOR. Luis: CONTADOR. María y José: CAPTURISTA. Pedro comienza como CONSULTA y termina como CAPTURISTA.

Las capturas siguientes se obtuvieron al ejecutar cada consulta o mutación en GraphiQL. El servidor consultó la lista vigente del rol antes de guardar. No se simularon respuestas.

Los rechazos se comprobaron también con SQL: no aumentaron las filas de cuentas, movimientos o relaciones. El cambio de puesto de Pedro no modificó los permisos del contador ni los de consulta.

La última sección incluye el SELECT real de MySQL. Las parejas CAPTURISTA/accounts.write y CONSULTA/concepts.write quedaron inactivas. La configuración activa coincide con la tabla del documento.

El código, instrucciones reproducibles, pruebas, capturas PNG, JSON HTTP y resultados SQL acompañan este reporte. No se incluyen contraseñas locales ni tokens. El envío al docente corresponde al autor.

00 Preparar el puesto inicial de Pedro

Actor: Ana. La demostración comienza con Pedro como CONSULTA.

Resultado: OK.

```
mutation ($u: ID!, $r: ID!) {
  assignRole(input: {companyUserId: $u, roleId: $r}) {
    id
    isActive
    role {
      code
    }
  }
}
```

GraphiQL

```
{
  "data": {
    "assignRole": {
      "id": "7",
      "isActive": true,
      "role": {
        "code": "CONSULTA"
      }
    }
  }
}
```

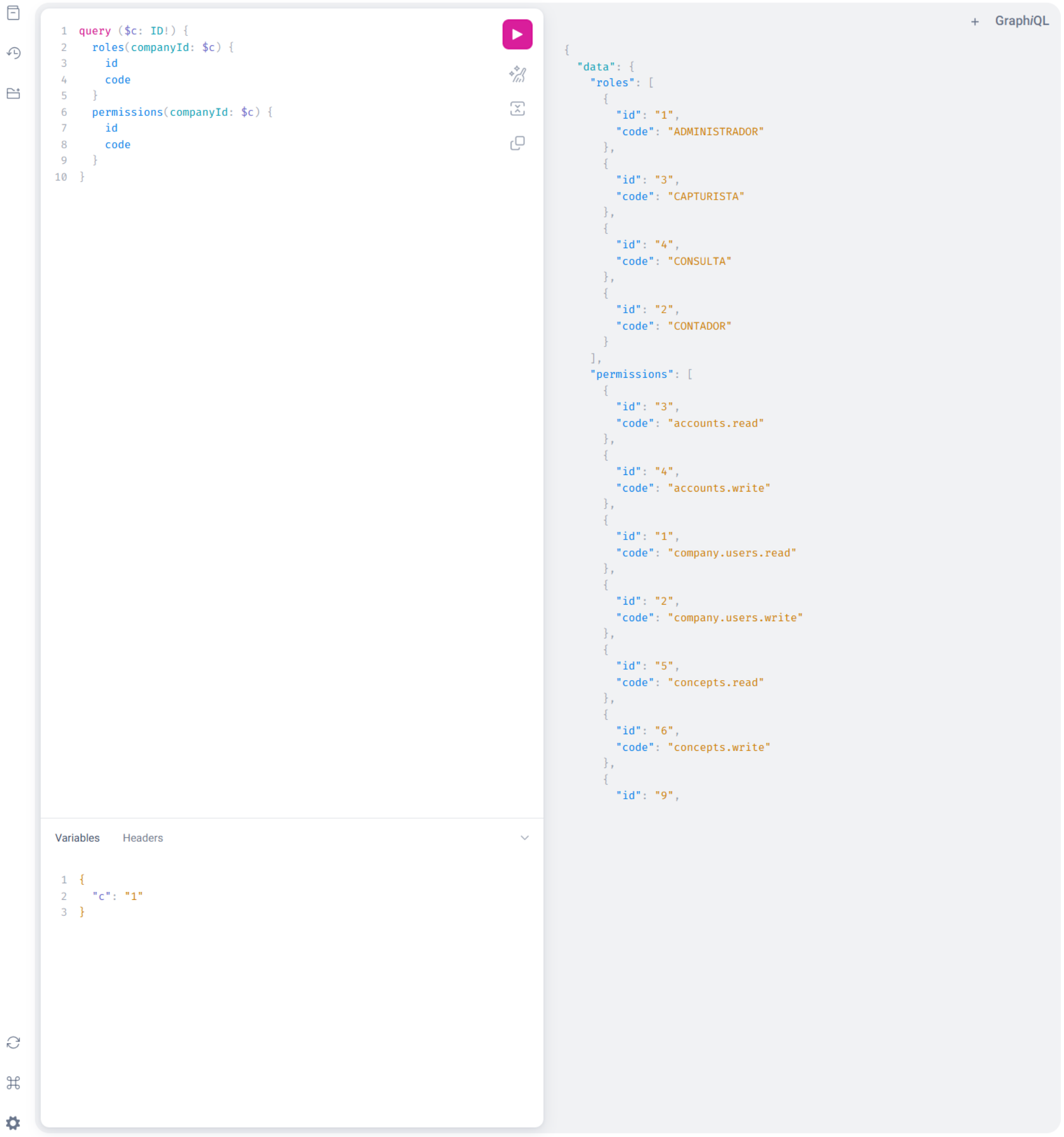
Variables Headers

```
{
  "u": "5",
  "r": "4"
}
```

01 Catálogos

Actor: Ana. La empresa tiene cuatro roles y diez permisos.

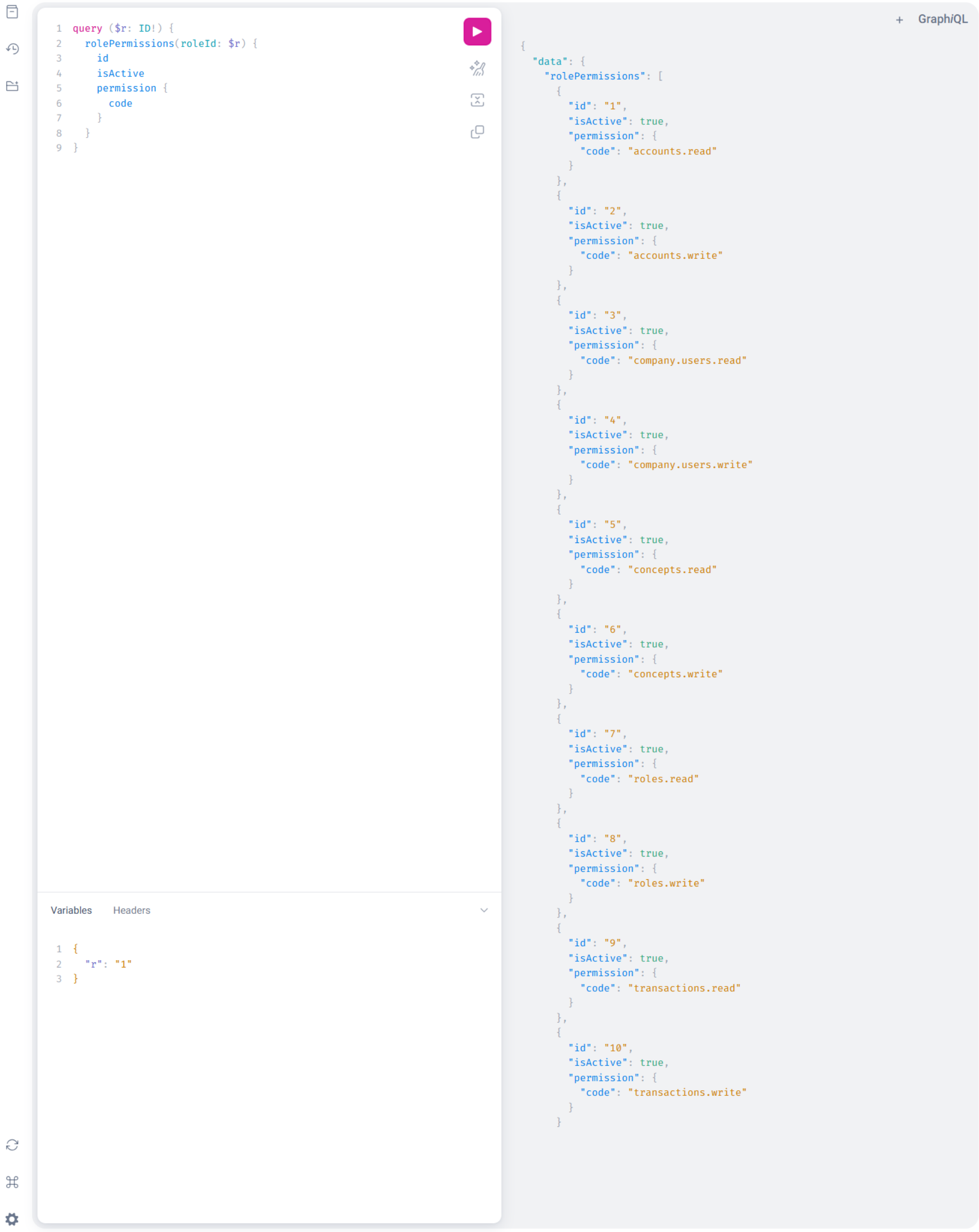
Resultado: OK.



02 Configuración inicial ADMINISTRADOR

Actor: Ana. Lista vigente del rol ADMINISTRADOR.

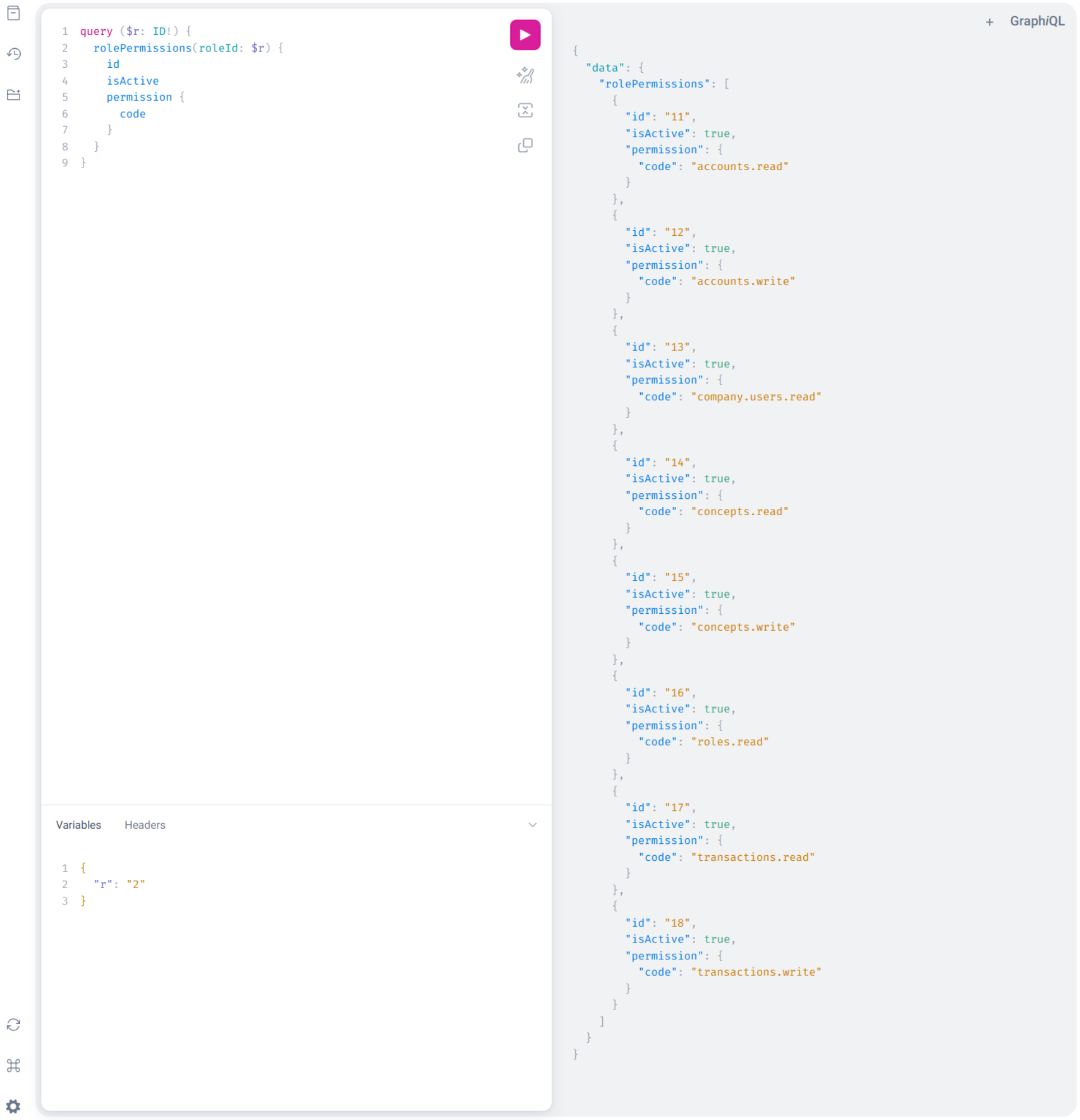
Resultado: OK.



02 Configuración inicial CONTADOR

Actor: Ana. Lista vigente del rol CONTADOR.

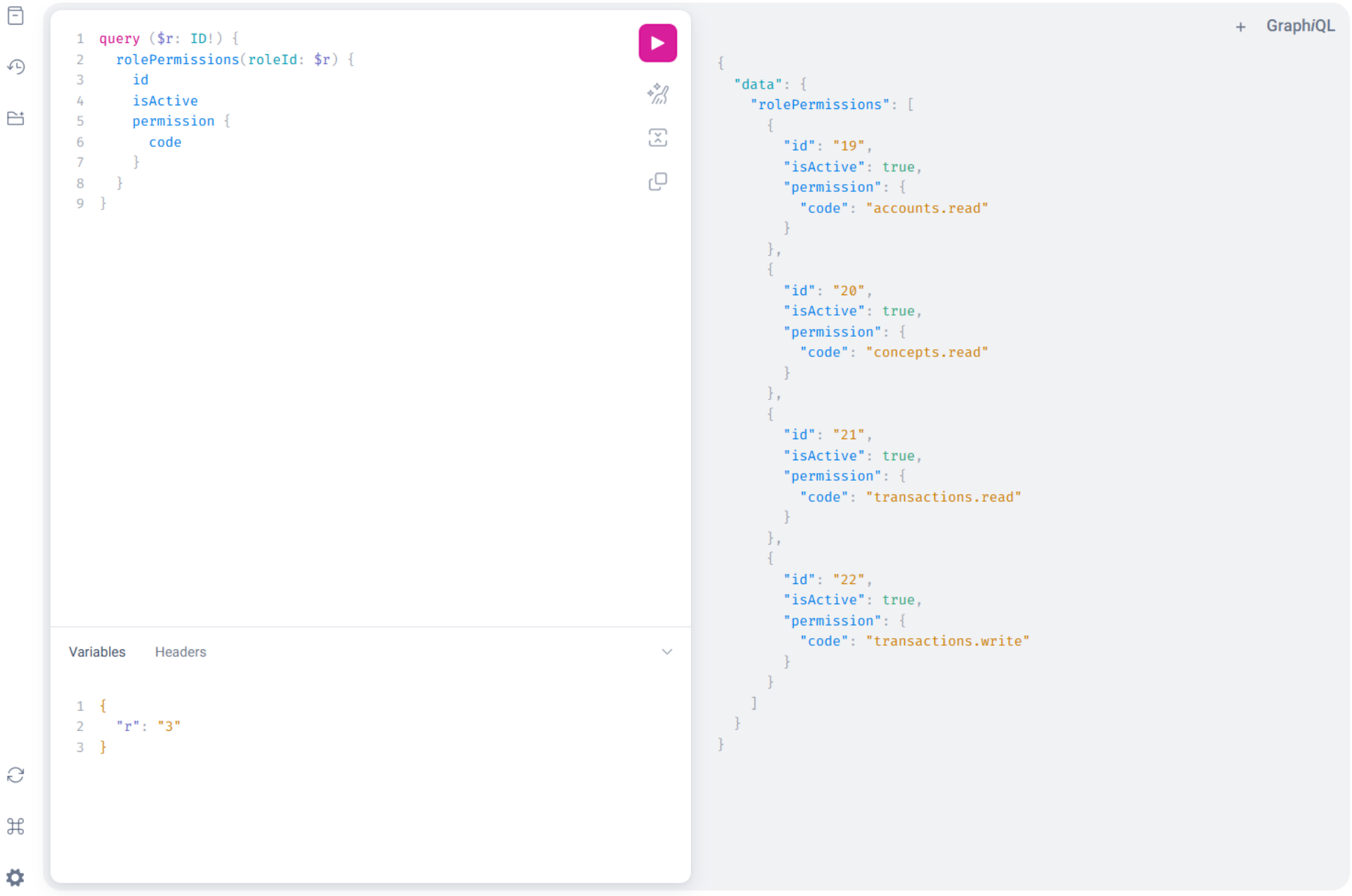
Resultado: OK.



02 Configuración inicial CAPTURISTA

Actor: Ana. Lista vigente del rol CAPTURISTA.

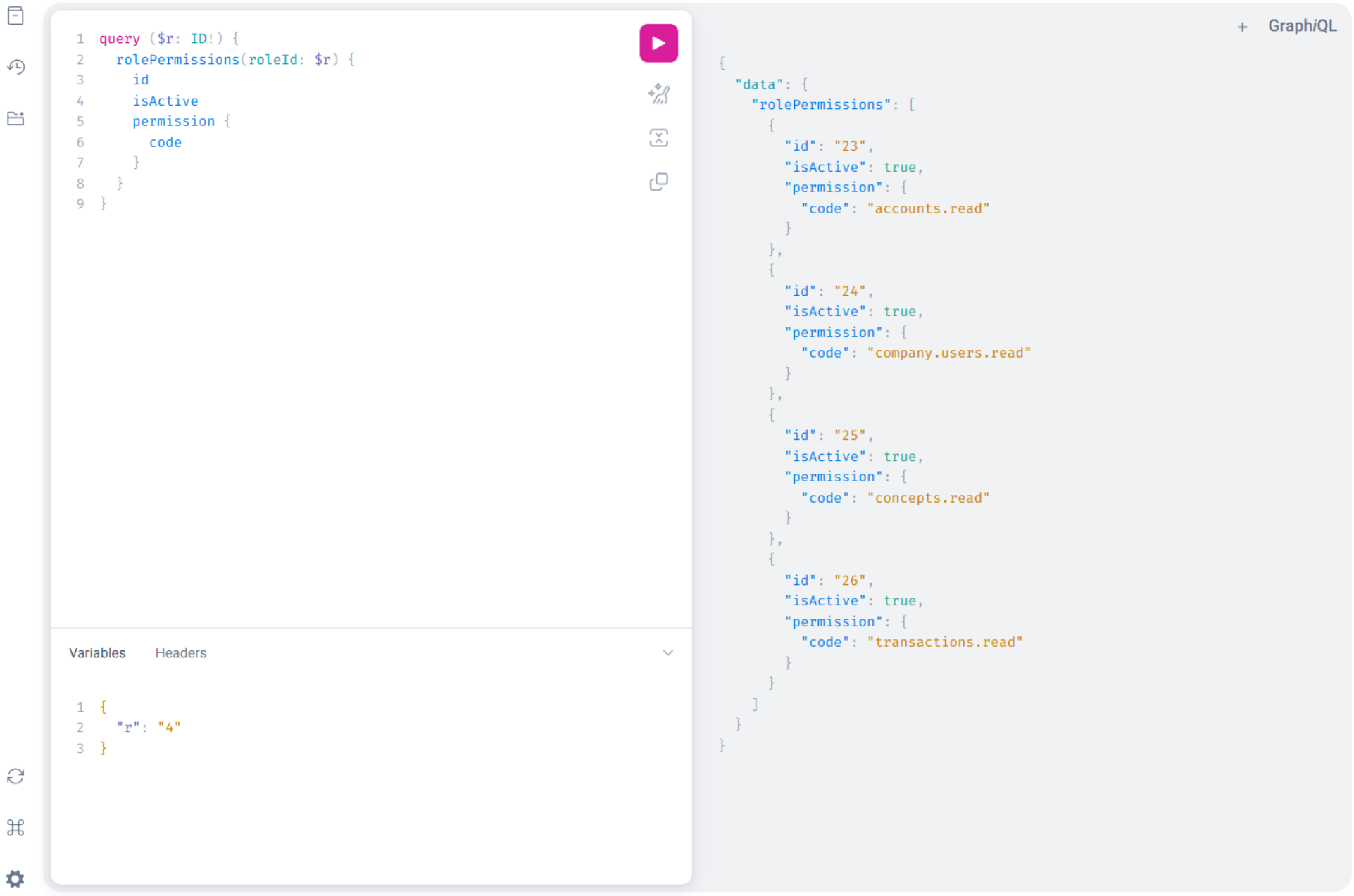
Resultado: OK.



02 Configuración inicial CONSULTA

Actor: Ana. Lista vigente del rol CONSULTA.

Resultado: OK.

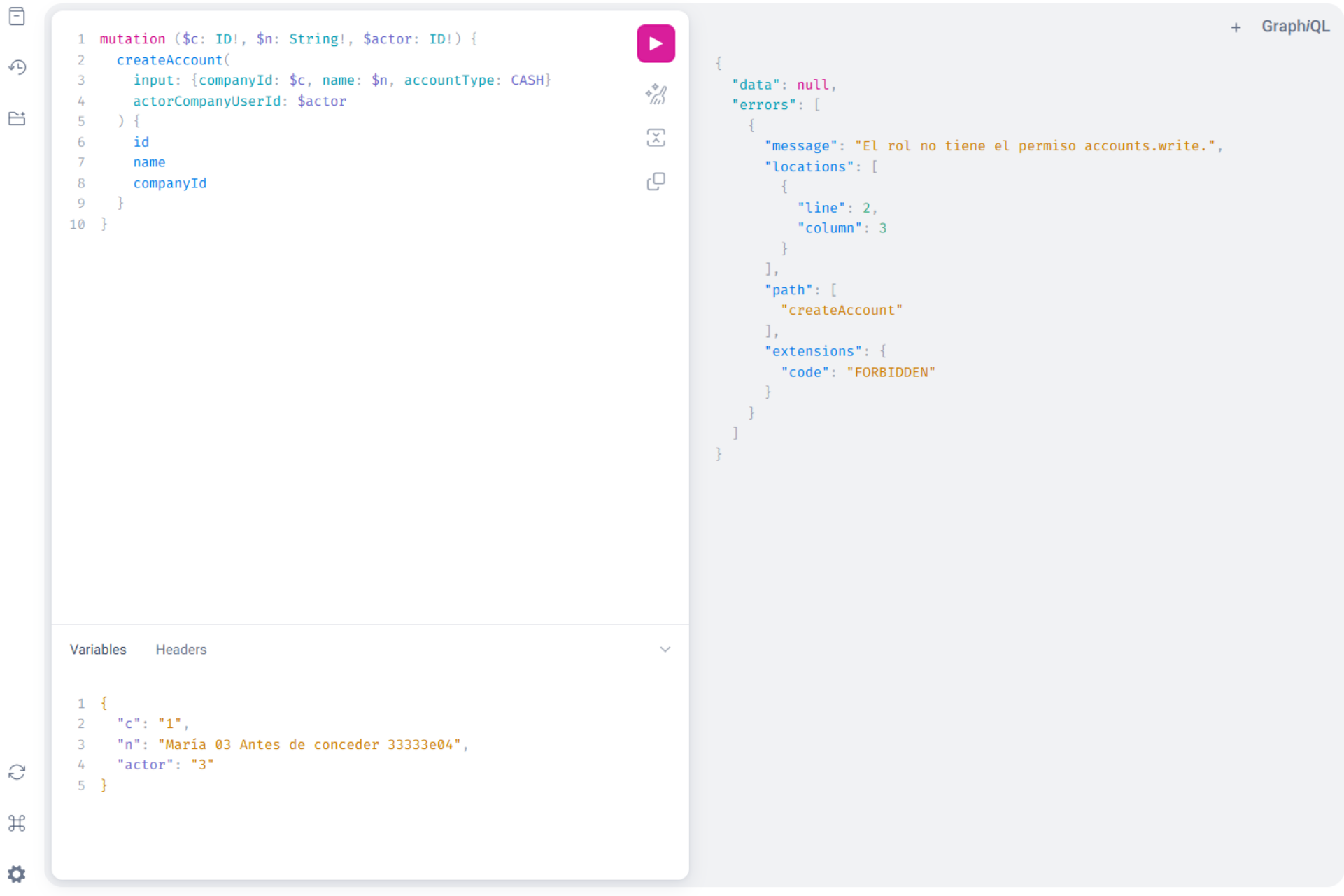


03 Antes de conceder Cuenta de María

Actor: María. El permiso accounts.write se decide por el rol vigente; un rechazo no inserta filas.

Resultado: FORBIDDEN.

Comprobación SQL: {"accounts_before": 7, "accounts_after": 7}

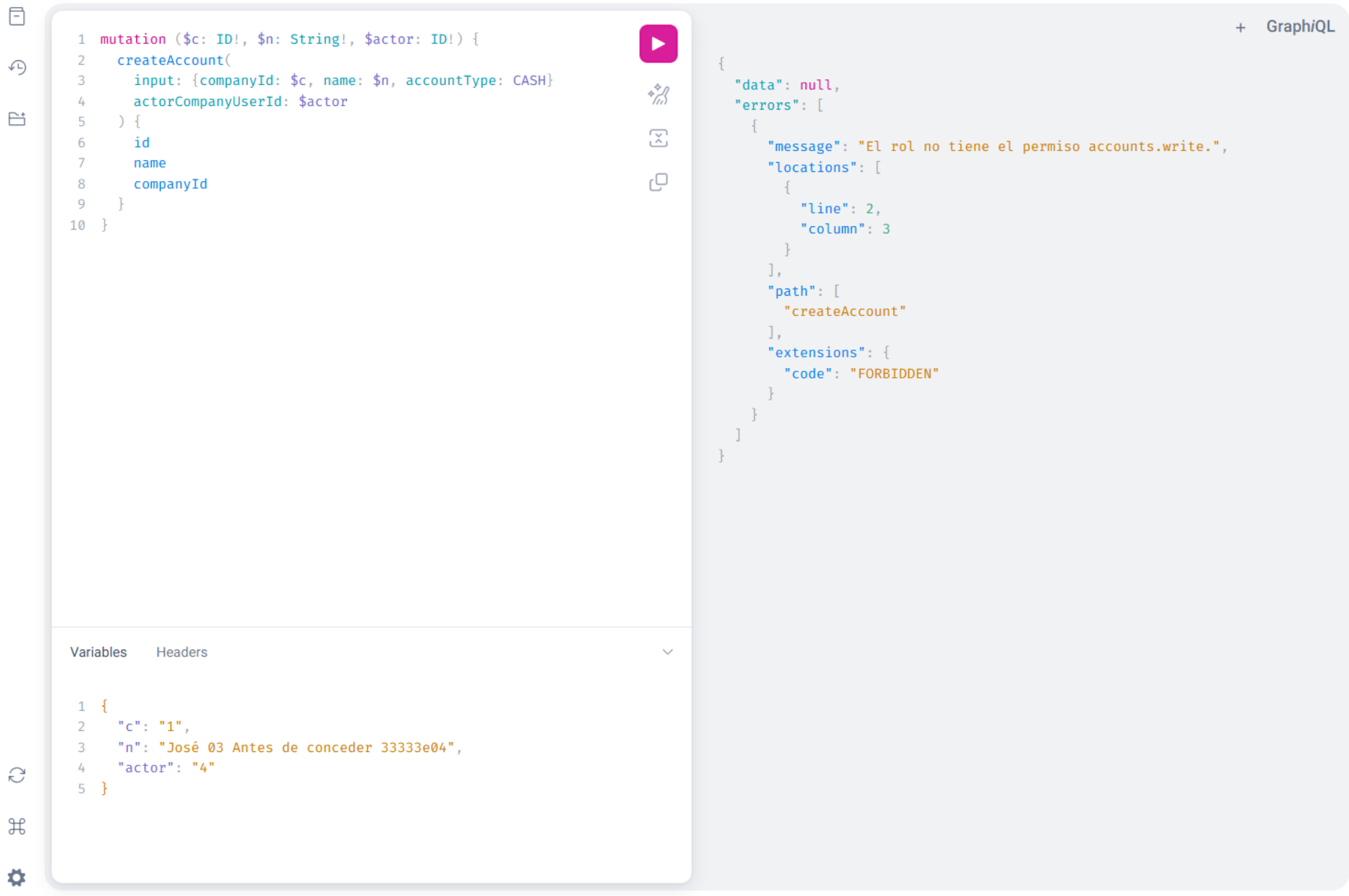


03 Antes de conceder Cuenta de José

Actor: José. El permiso accounts.write se decide por el rol vigente; un rechazo no inserta filas.

Resultado: FORBIDDEN.

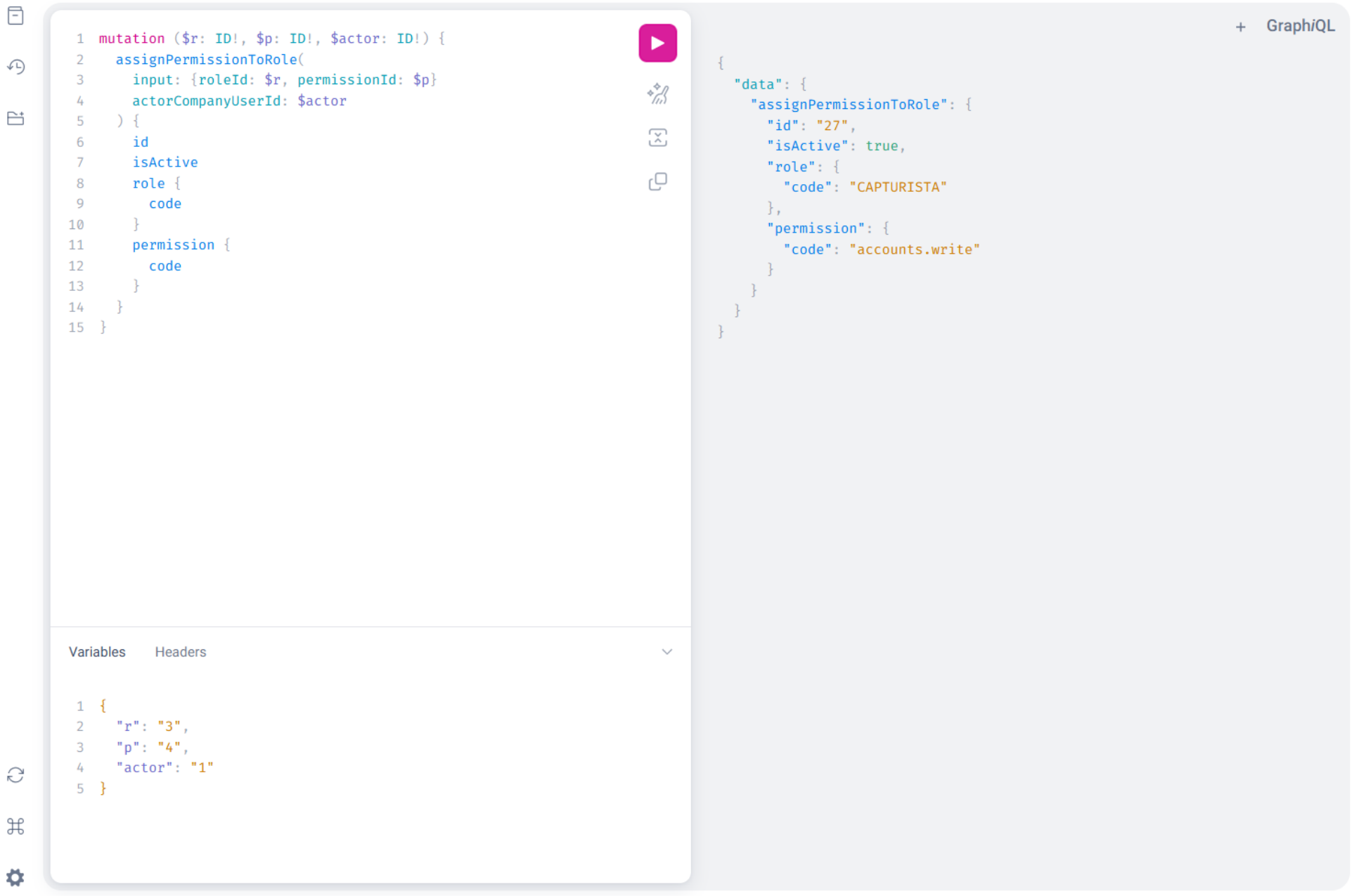
Comprobación SQL: {"accounts_before": 7, "accounts_after": 7}



04 Ana concede accounts.write

Actor: Ana. Un cambio en CAPTURISTA beneficia a sus dos integrantes.

Resultado: OK.

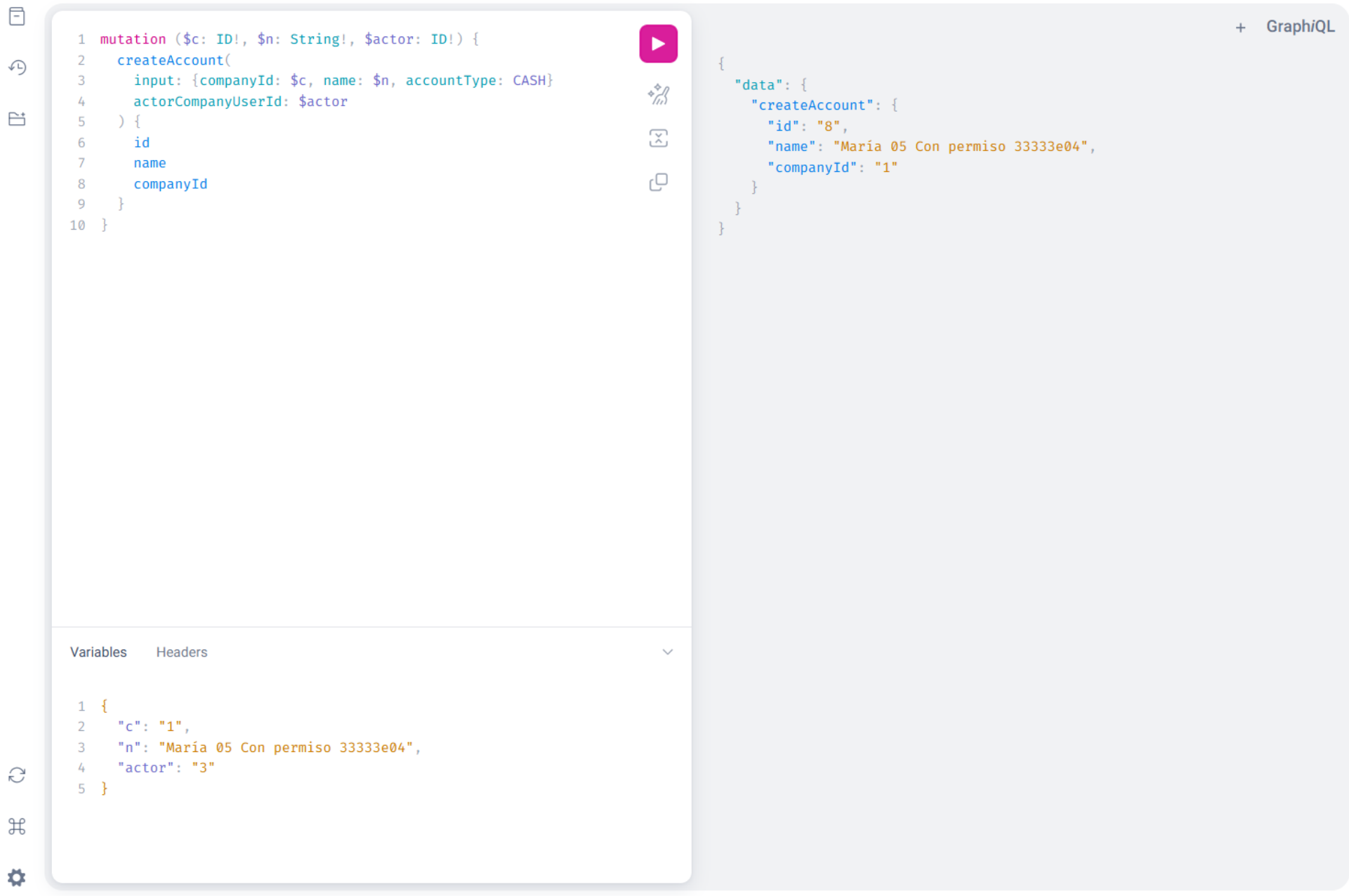


05 Con permiso Cuenta de María

Actor: María. El permiso accounts.write se decide por el rol vigente; un rechazo no inserta filas.

Resultado: OK.

Comprobación SQL: {"accounts_before": 7, "accounts_after": 8}

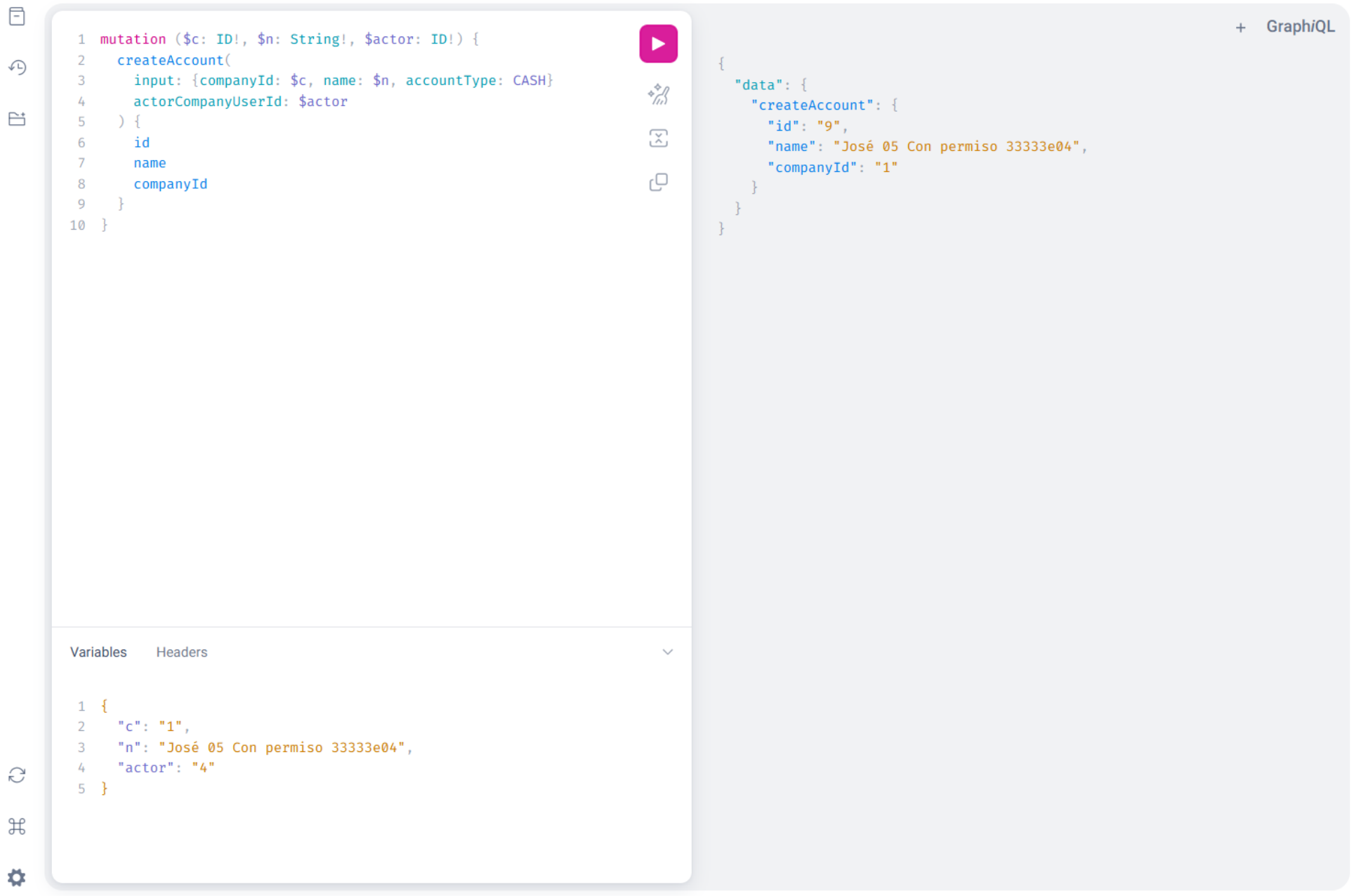


05 Con permiso Cuenta de José

Actor: José. El permiso accounts.write se decide por el rol vigente; un rechazo no inserta filas.

Resultado: OK.

Comprobación SQL: {"accounts_before": 8, "accounts_after": 9}

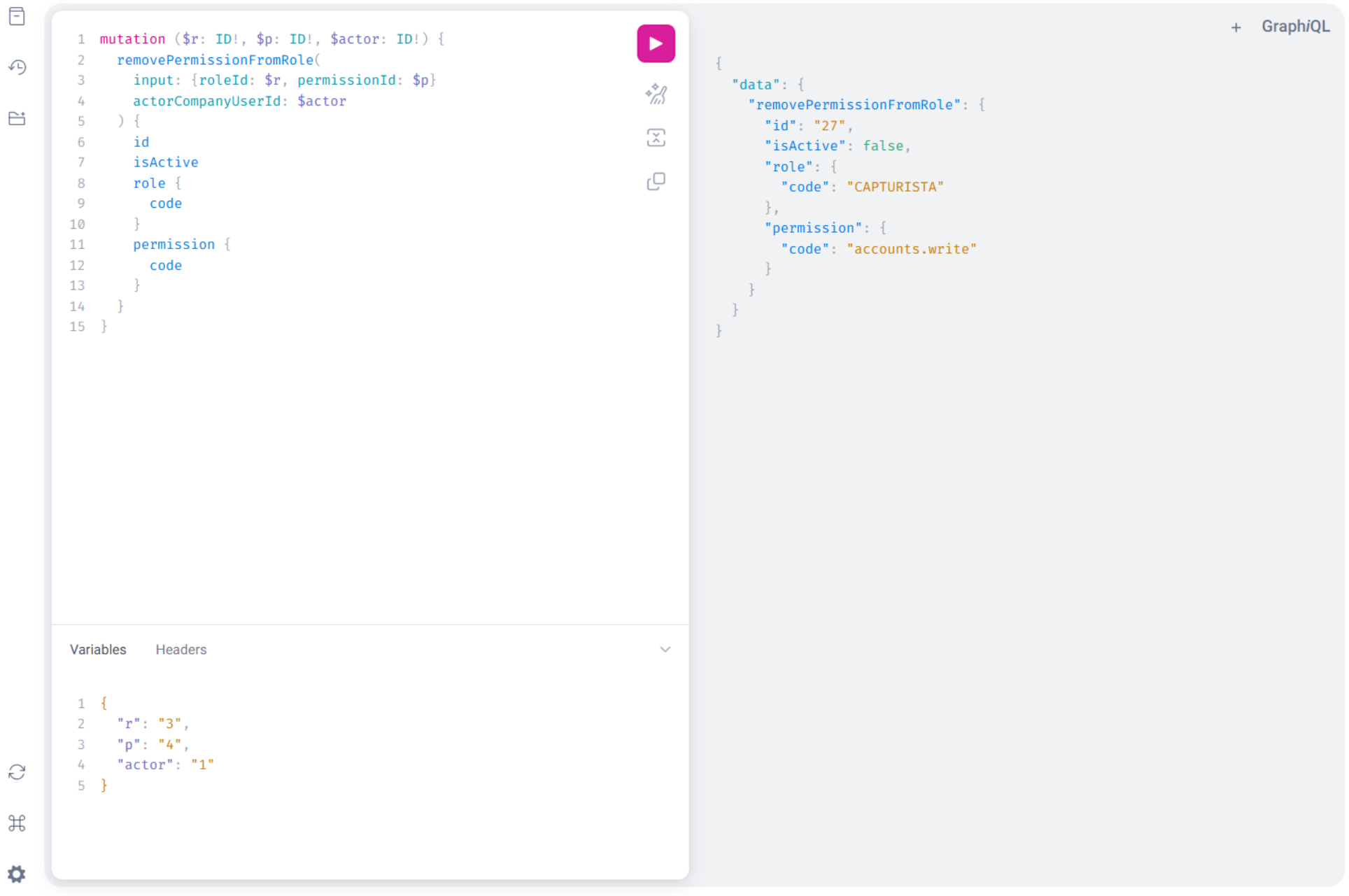


06 Ana quita accounts.write

Actor: Ana. Quitar un permiso desactiva la pareja y conserva el ID.

Resultado: OK.

Comprobación SQL: {"role_permission_id": 27, "is_active": false, "row_preserved": true}



07 Tras quitar permiso Cuenta de María

Actor: María. El permiso accounts.write se decide por el rol vigente; un rechazo no inserta filas.

Resultado: FORBIDDEN.

Comprobación SQL: {"accounts_before": 9, "accounts_after": 9}

```
mutation ($c: ID!, $n: String!, $actor: ID!) {
  createAccount(
    input: {companyId: $c, name: $n, accountType: CASH}
    actorCompanyUserId: $actor
  ) {
    id
    name
    companyId
  }
}
```

GraphiQL

```
{
  "data": null,
  "errors": [
    {
      "message": "El rol no tiene el permiso accounts.write.",
      "locations": [
        {
          "line": 2,
          "column": 3
        }
      ],
      "path": [
        "createAccount"
      ],
      "extensions": {
        "code": "FORBIDDEN"
      }
    }
  ]
}
```

Variables Headers

```
{
  "c": "1",
  "n": "María 07 Tras quitar permiso 33333e04",
  "actor": "3"
}
```

07 Tras quitar permiso Cuenta de José

Actor: José. El permiso accounts.write se decide por el rol vigente; un rechazo no inserta filas.

Resultado: FORBIDDEN.

Comprobación SQL: {"accounts_before": 9, "accounts_after": 9}

```
mutation ($c: ID!, $n: String!, $actor: ID!) {
  createAccount(
    input: {companyId: $c, name: $n, accountType: CASH}
    actorCompanyUserId: $actor
  ) {
    id
    name
    companyId
  }
}
```

GraphiQL

```
{
  "data": null,
  "errors": [
    {
      "message": "El rol no tiene el permiso accounts.write.",
      "locations": [
        {
          "line": 2,
          "column": 3
        }
      ],
      "path": [
        "createAccount"
      ],
      "extensions": {
        "code": "FORBIDDEN"
      }
    }
  ]
}
```

Variables Headers

```
{
  "c": "1",
  "n": "José 07 Tras quitar permiso 33333e04",
  "actor": "4"
}
```

08 Pedro como CONSULTA

Actor: Pedro. CONSULTA no tiene transactions.write; el rechazo no crea movimientos.

Resultado: FORBIDDEN.

Comprobación SQL: {"transactions_before": 2, "transactions_after": 2}

```
mutation ($a: ID!, $c: ID!, $actor: ID!) {
  createTransaction(
    input: {accountId: $a, conceptId: $c, transactionType: INCOM
    actorCompanyUserId: $actor
  ) {
    id
    amount
    capturedBy
  }
}
```

GraphiQL

```
{
  "data": null,
  "errors": [
    {
      "message": "El rol no tiene el permiso transactions.write.",
      "locations": [
        {
          "line": 2,
          "column": 3
        }
      ],
      "path": [
        "createTransaction"
      ],
      "extensions": {
        "code": "FORBIDDEN"
      }
    }
  ]
}
```

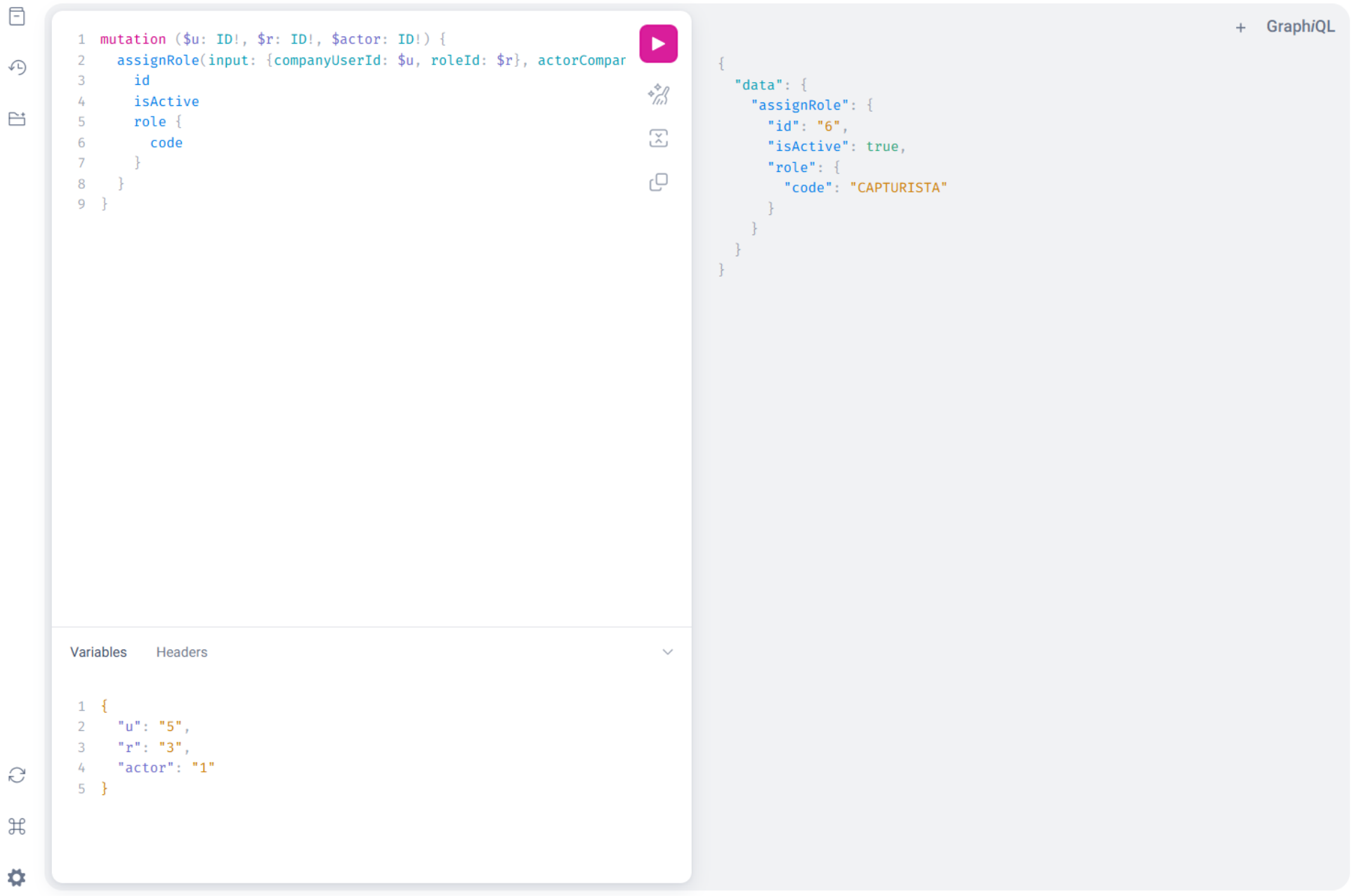
Variables Headers

```
{
  "a": "1",
  "c": "1",
  "actor": "5"
}
```

09 Ana cambia el puesto de Pedro

Actor: Ana. assignRole deja un solo rol activo y conserva el anterior inactivo.

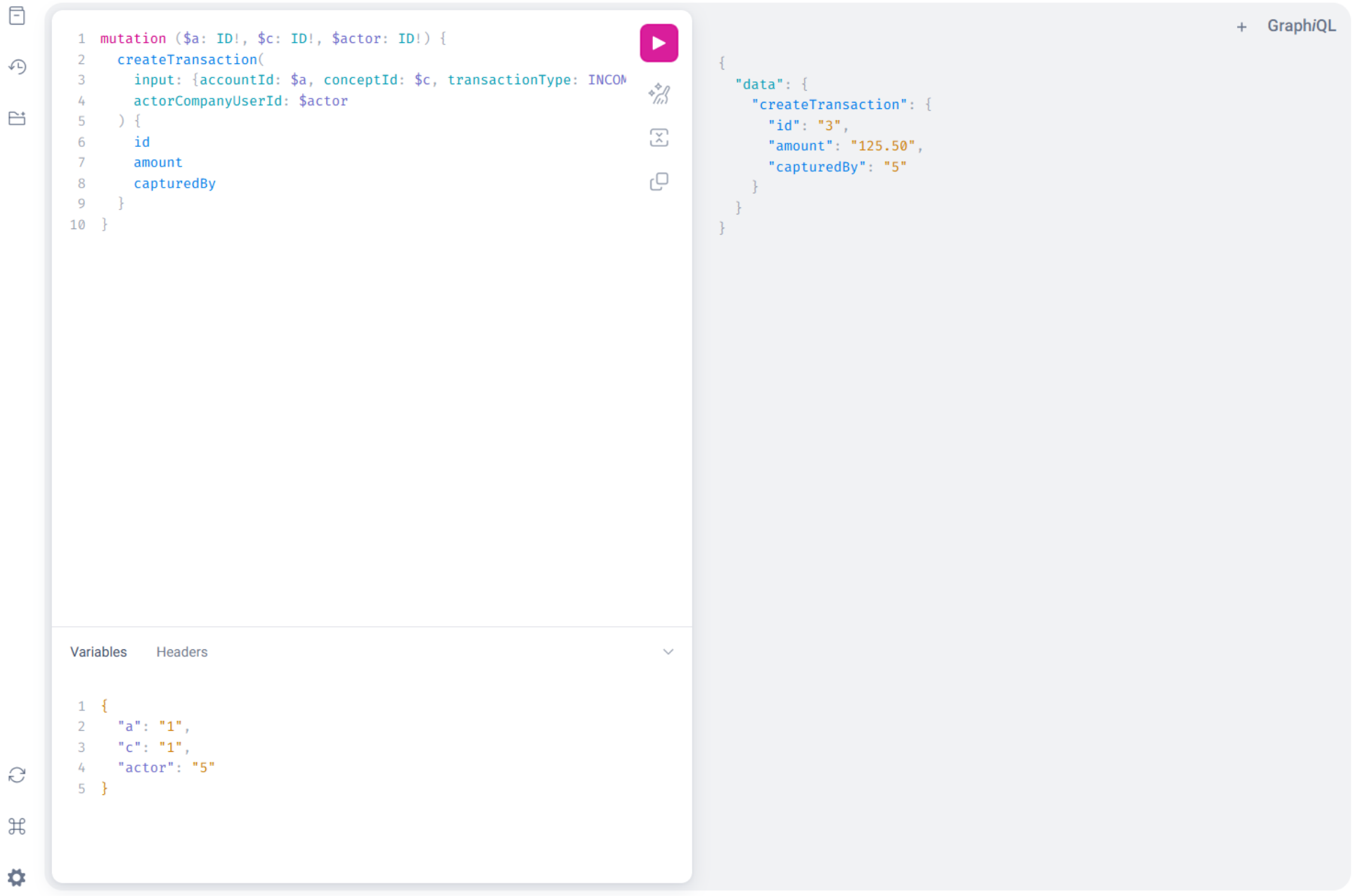
Resultado: OK.



10 Pedro como CAPTURISTA

Actor: Pedro. El mismo token recibe los permisos del nuevo rol.

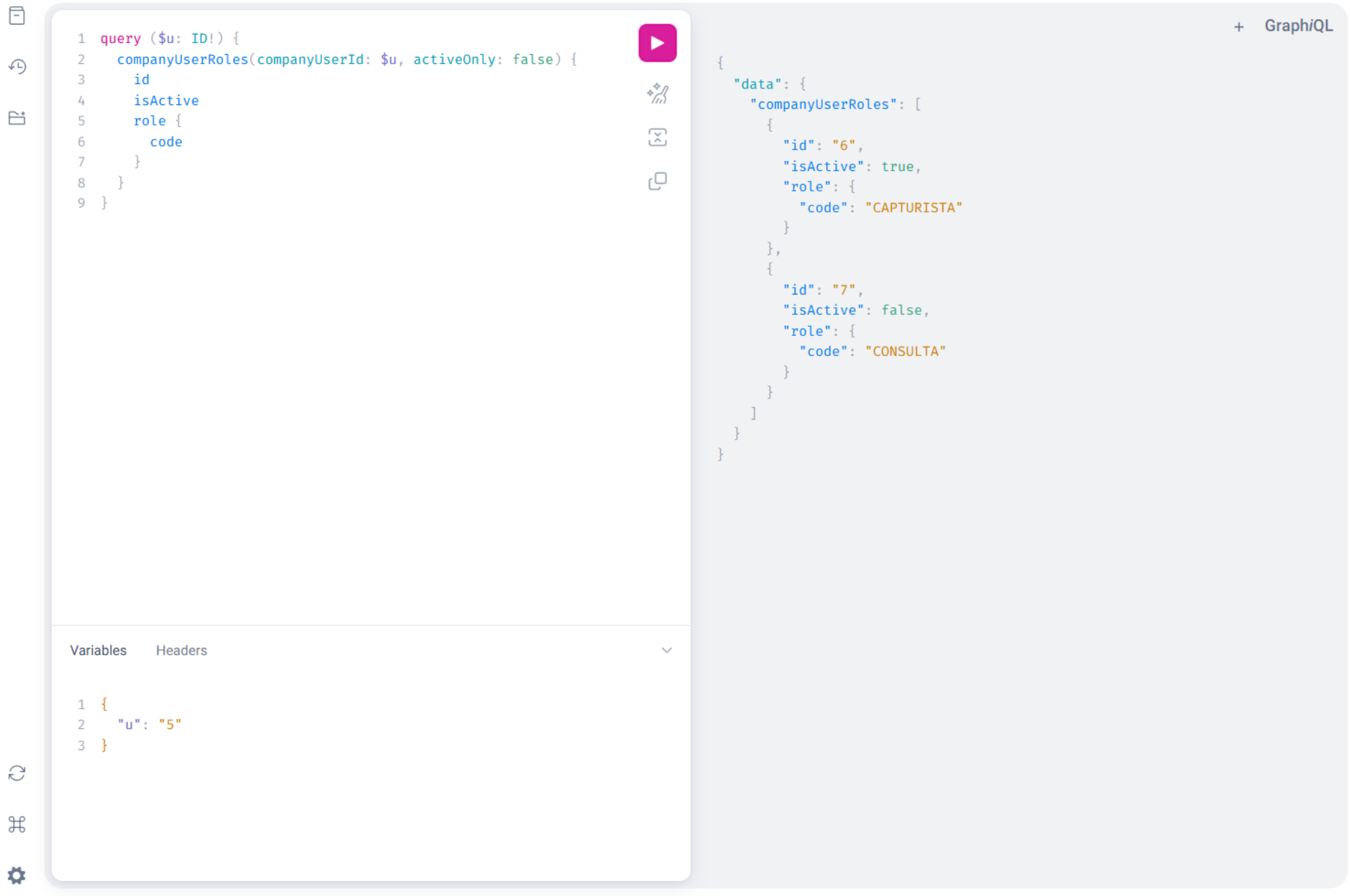
Resultado: OK.



11 Historial de Pedro

Actor: Ana. Pedro tiene CAPTURISTA activo y CONSULTA inactivo.

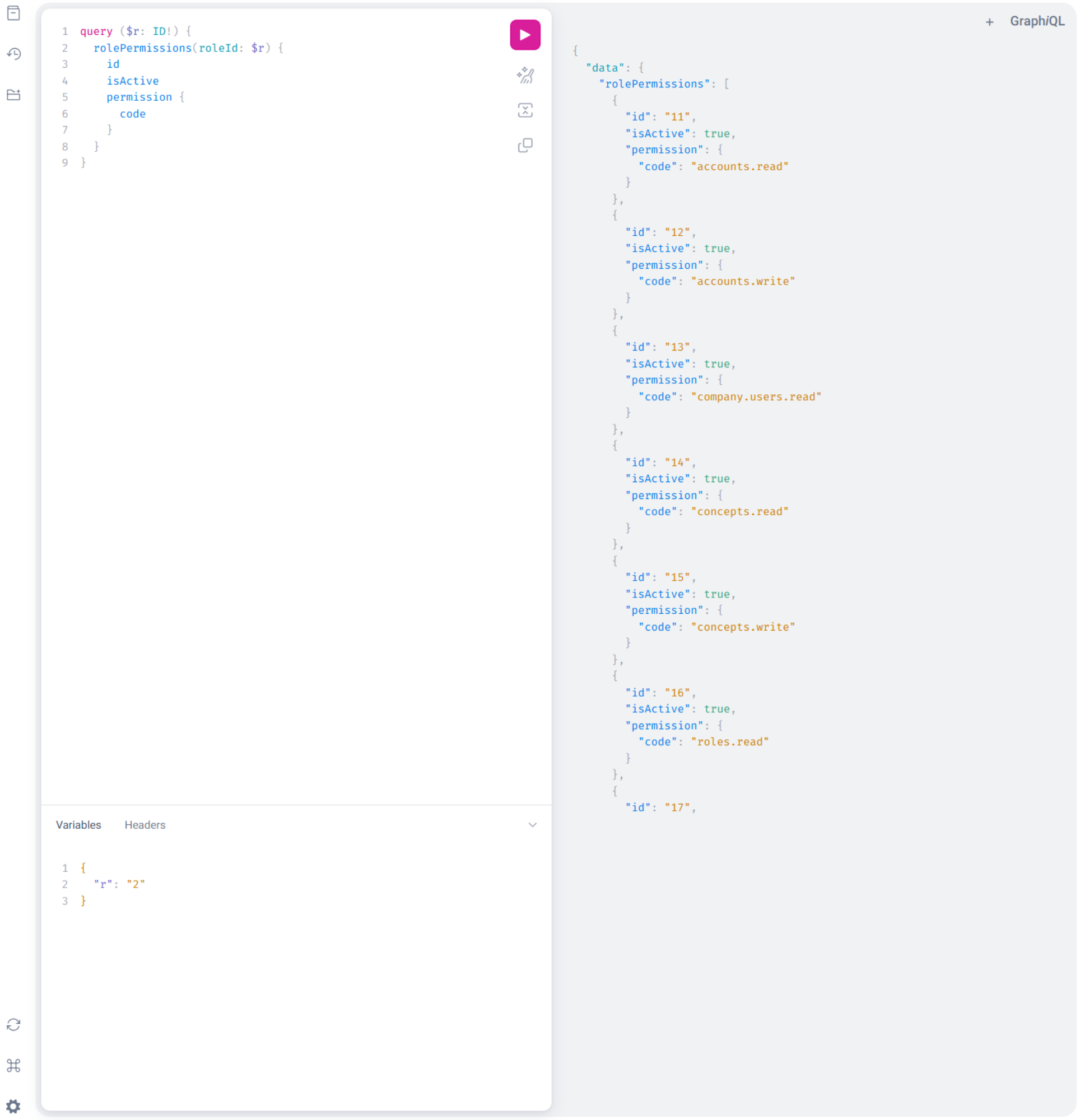
Resultado: OK.



12 Contador sin cambios

Actor: Ana. Lista vigente del rol CONTADOR.

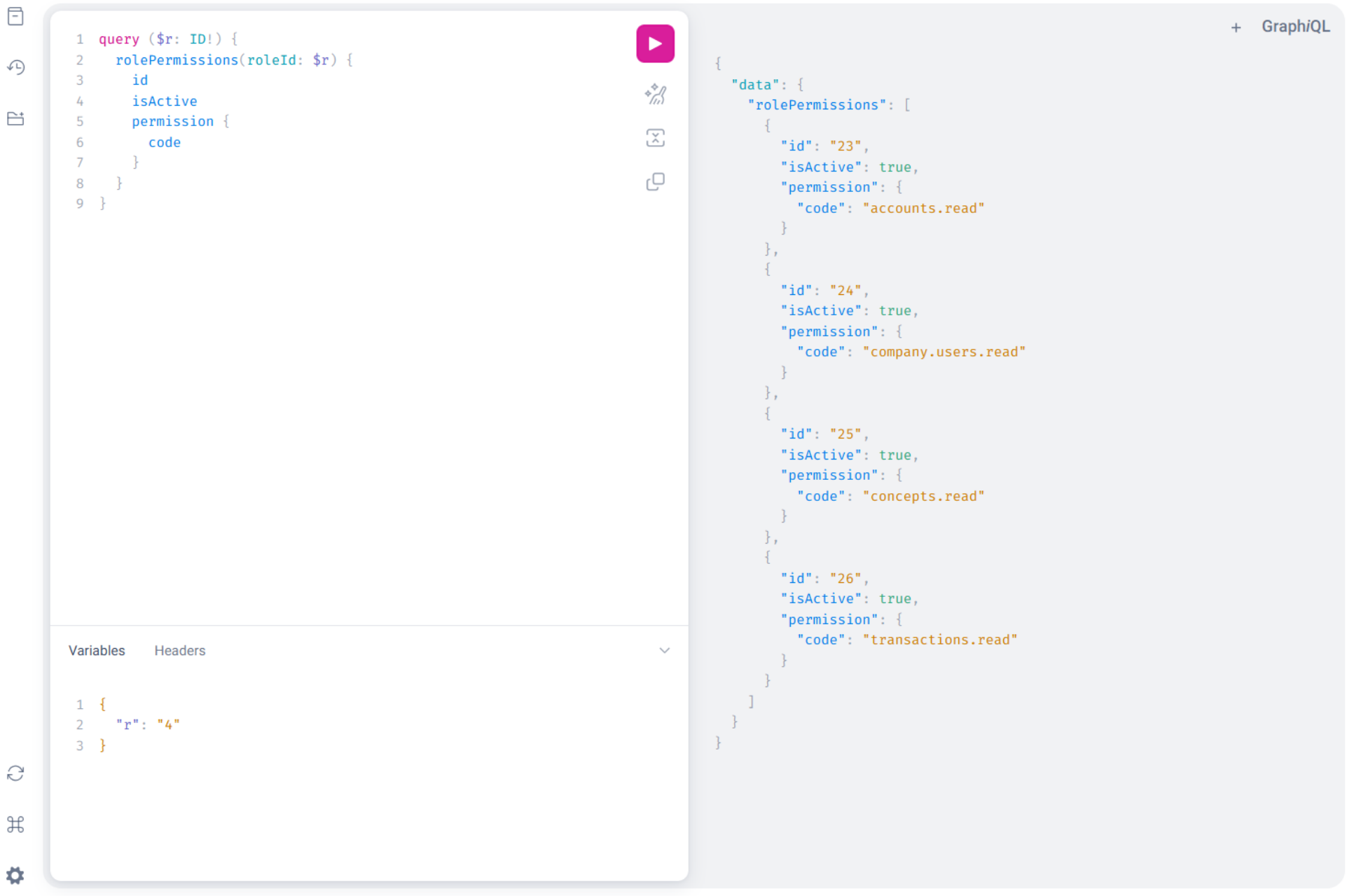
Resultado: OK.



13 Consulta sin cambios

Actor: Ana. Lista vigente del rol CONSULTA.

Resultado: OK.

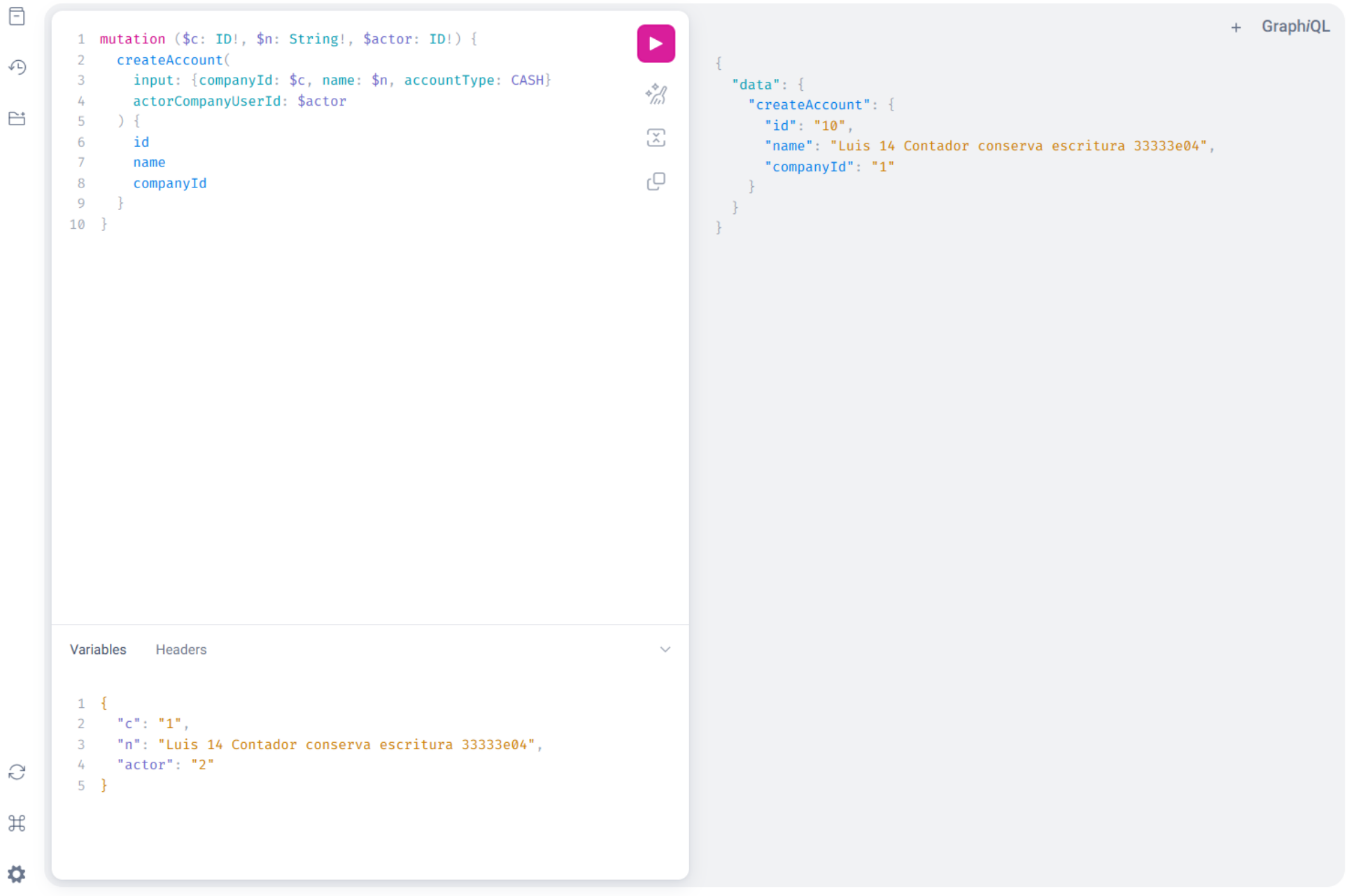


14 Contador conserva escritura Cuenta de Luis

Actor: Luis. El permiso accounts.write se decide por el rol vigente; un rechazo no inserta filas.

Resultado: OK.

Comprobación SQL: {"accounts_before": 9, "accounts_after": 10}



15 José intenta configurar permisos

Actor: José. Sin roles.write no se cambia la lista ni se inserta la pareja.

Resultado: FORBIDDEN.

Comprobación SQL: {"role_permissions_before": 28, "role_permissions_after": 28}

```
mutation ($r: ID!, $p: ID!, $actor: ID!) {
  assignPermissionToRole(
    input: {roleId: $r, permissionId: $p}
    actorCompanyUserId: $actor
  ) {
    id
    isActive
    role {
      code
    }
    permission {
      code
    }
  }
}
```

Variables Headers

```
{
  "r": "4",
  "p": "6",
  "actor": "4"
}
```

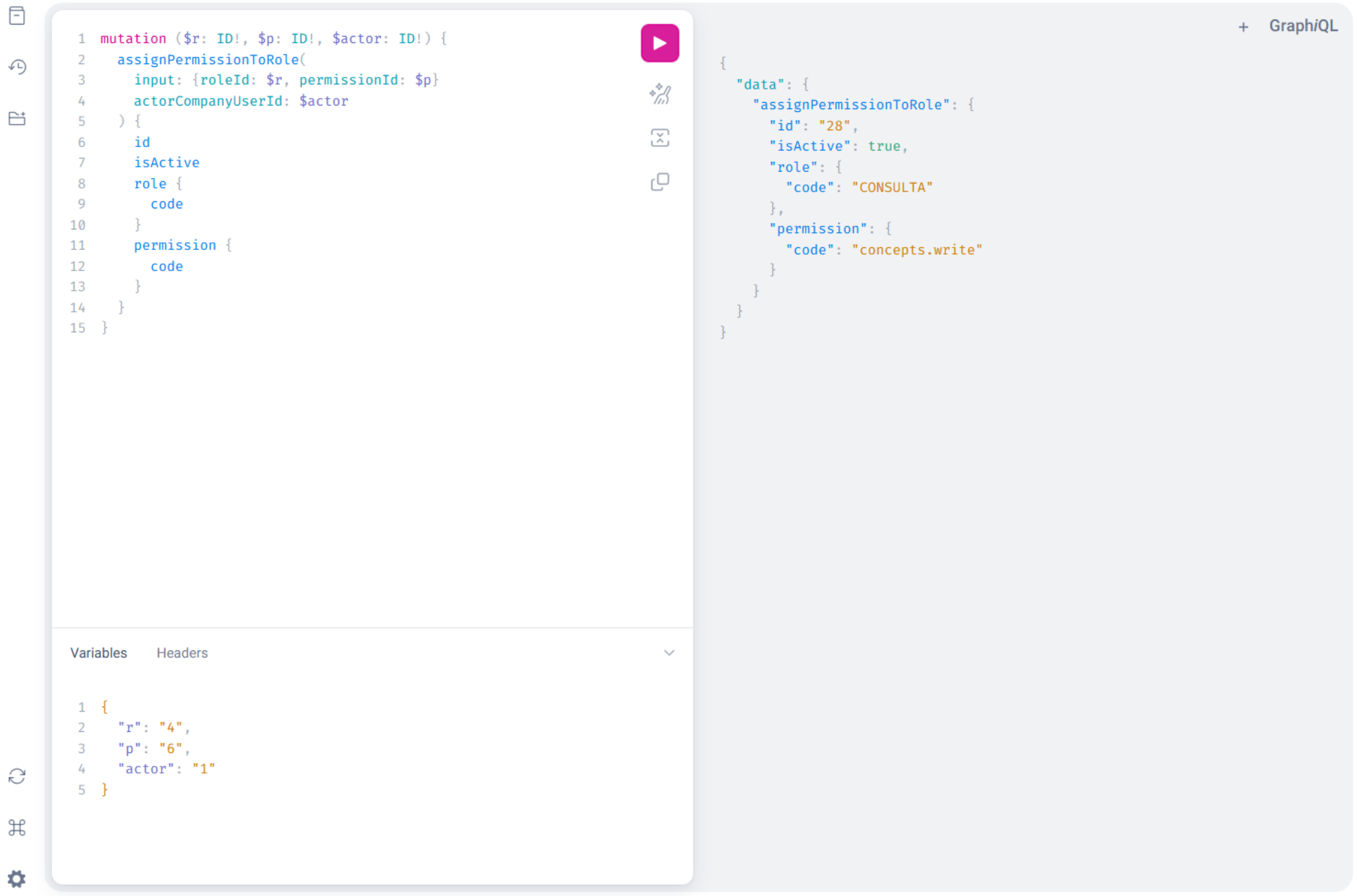
GraphiQL

```
{
  "data": null,
  "errors": [
    {
      "message": "El rol no tiene el permiso roles.write.",
      "locations": [
        {
          "line": 2,
          "column": 3
        }
      ],
      "path": [
        "assignPermissionToRole"
      ],
      "extensions": {
        "code": "FORBIDDEN"
      }
    }
  ]
}
```

16 Ana agrega concepts.write a CONSULTA

Actor: Ana. Con roles.write la misma operación sí se permite.

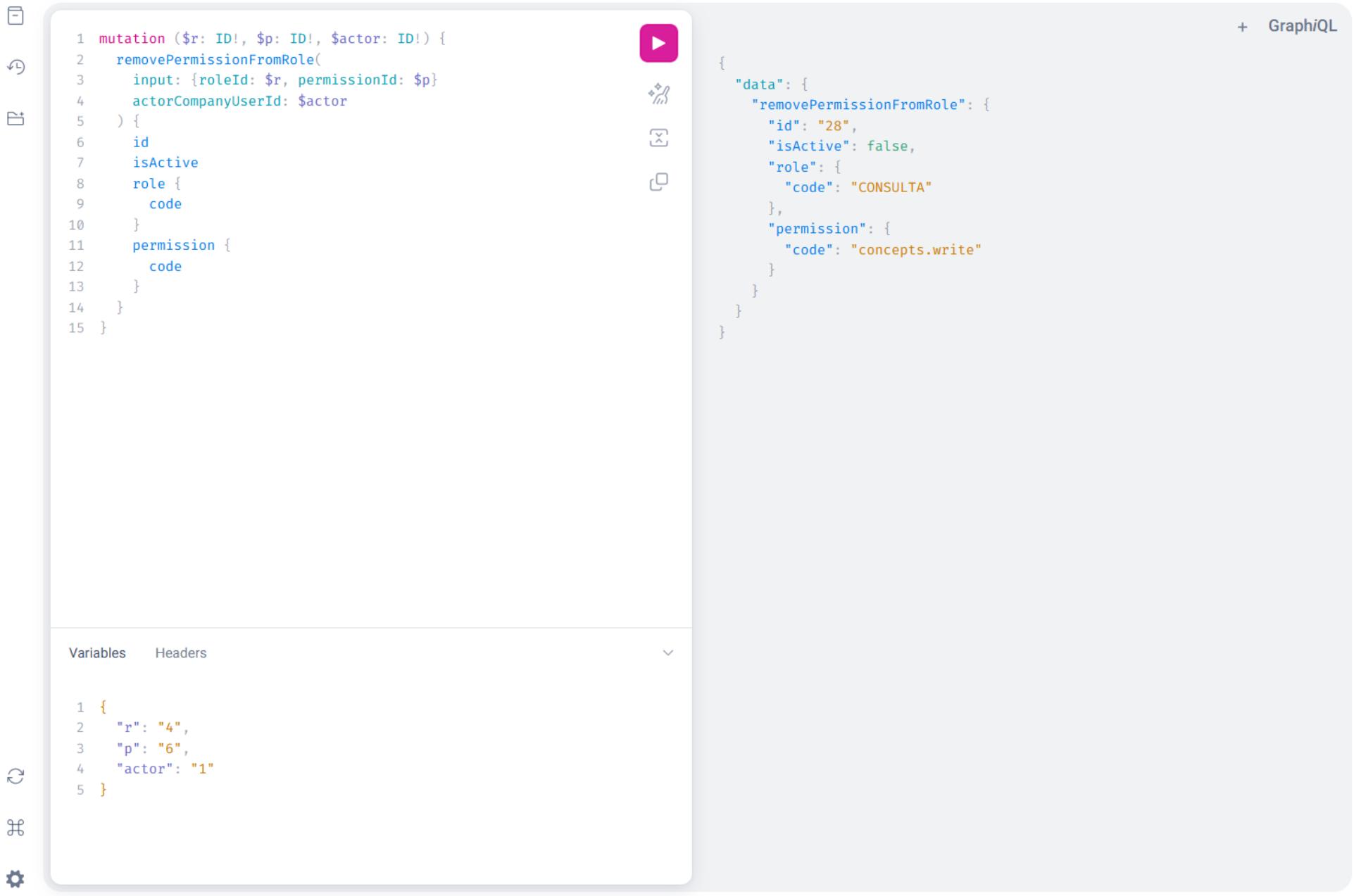
Resultado: OK.



17 Ana restaura CONSULTA

Actor: Ana. La pareja queda inactiva y CONSULTA vuelve a ser de lectura.

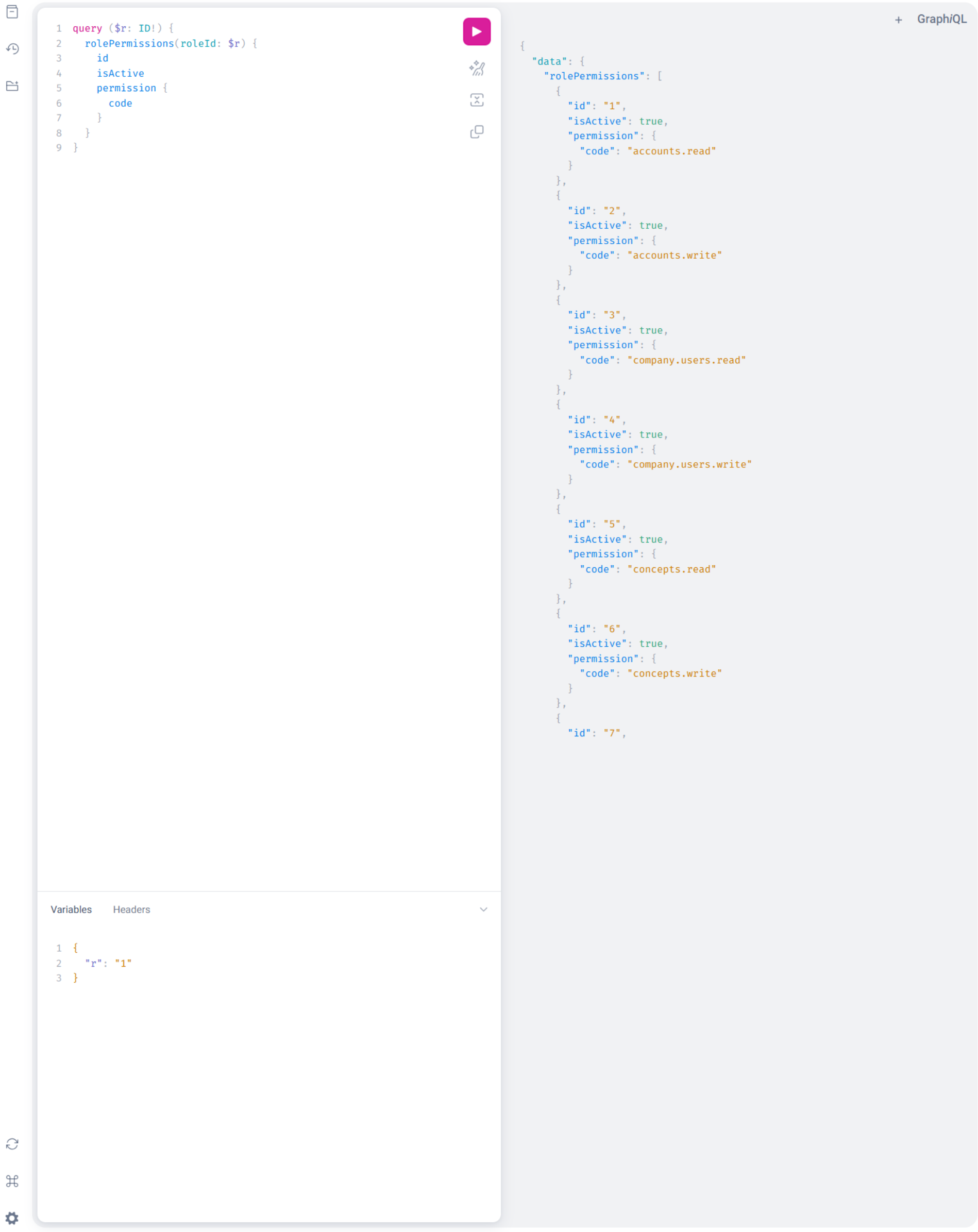
Resultado: OK.



18 Matriz final ADMINISTRADOR

Actor: Ana. Lista vigente del rol ADMINISTRADOR.

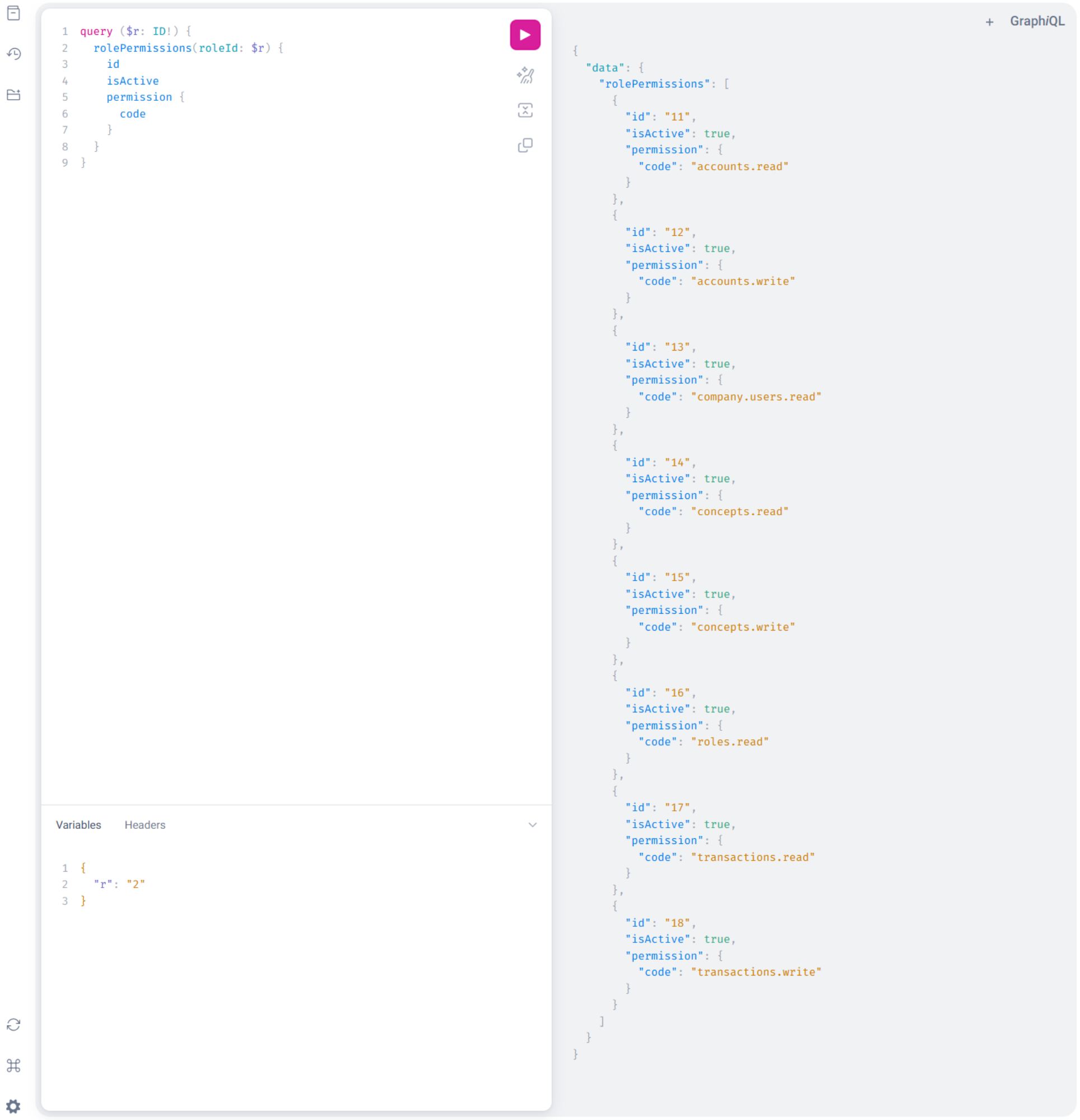
Resultado: OK.



18 Matriz final CONTADOR

Actor: Ana. Lista vigente del rol CONTADOR.

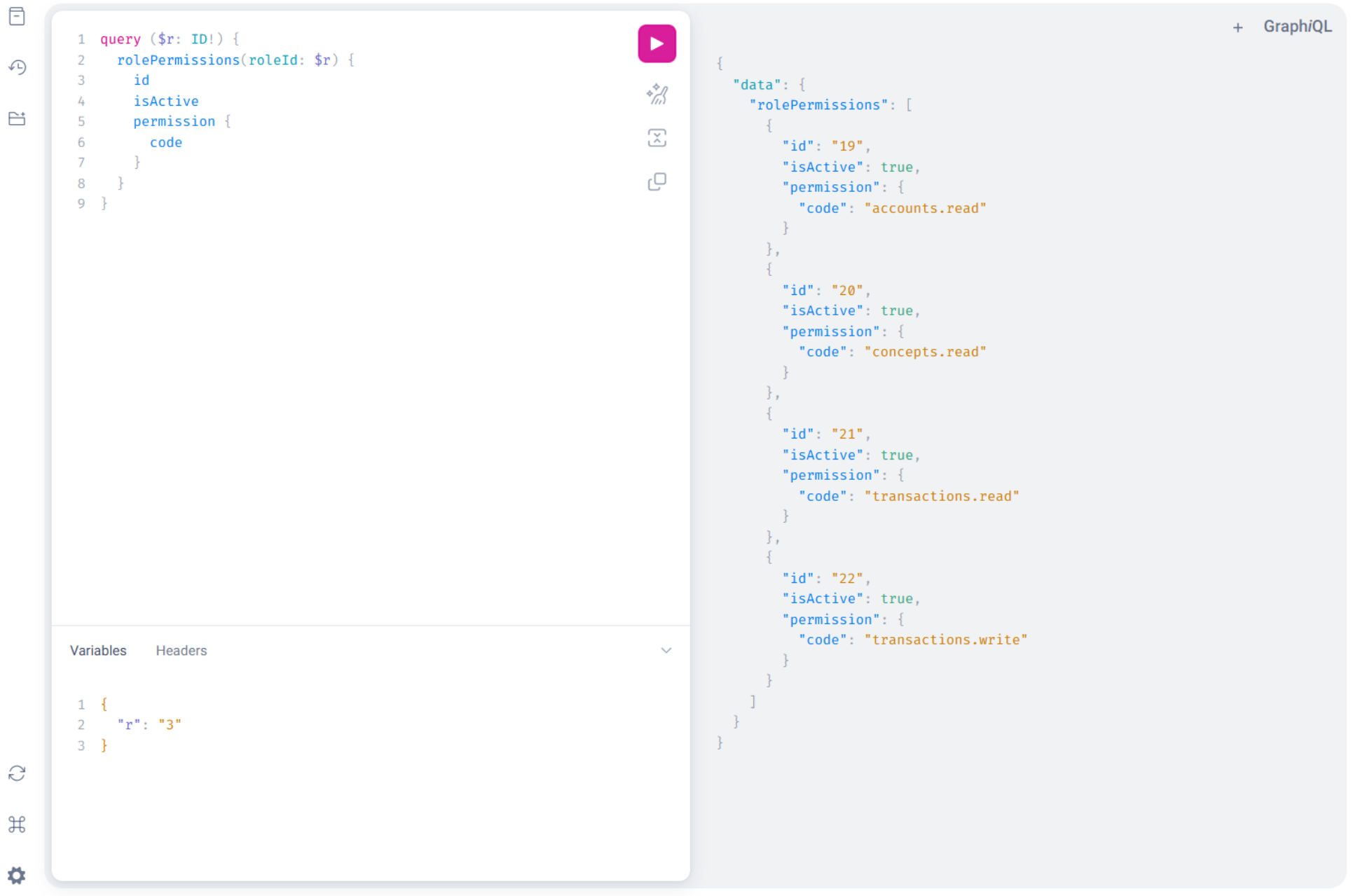
Resultado: OK.



18 Matriz final CAPTURISTA

Actor: Ana. Lista vigente del rol CAPTURISTA.

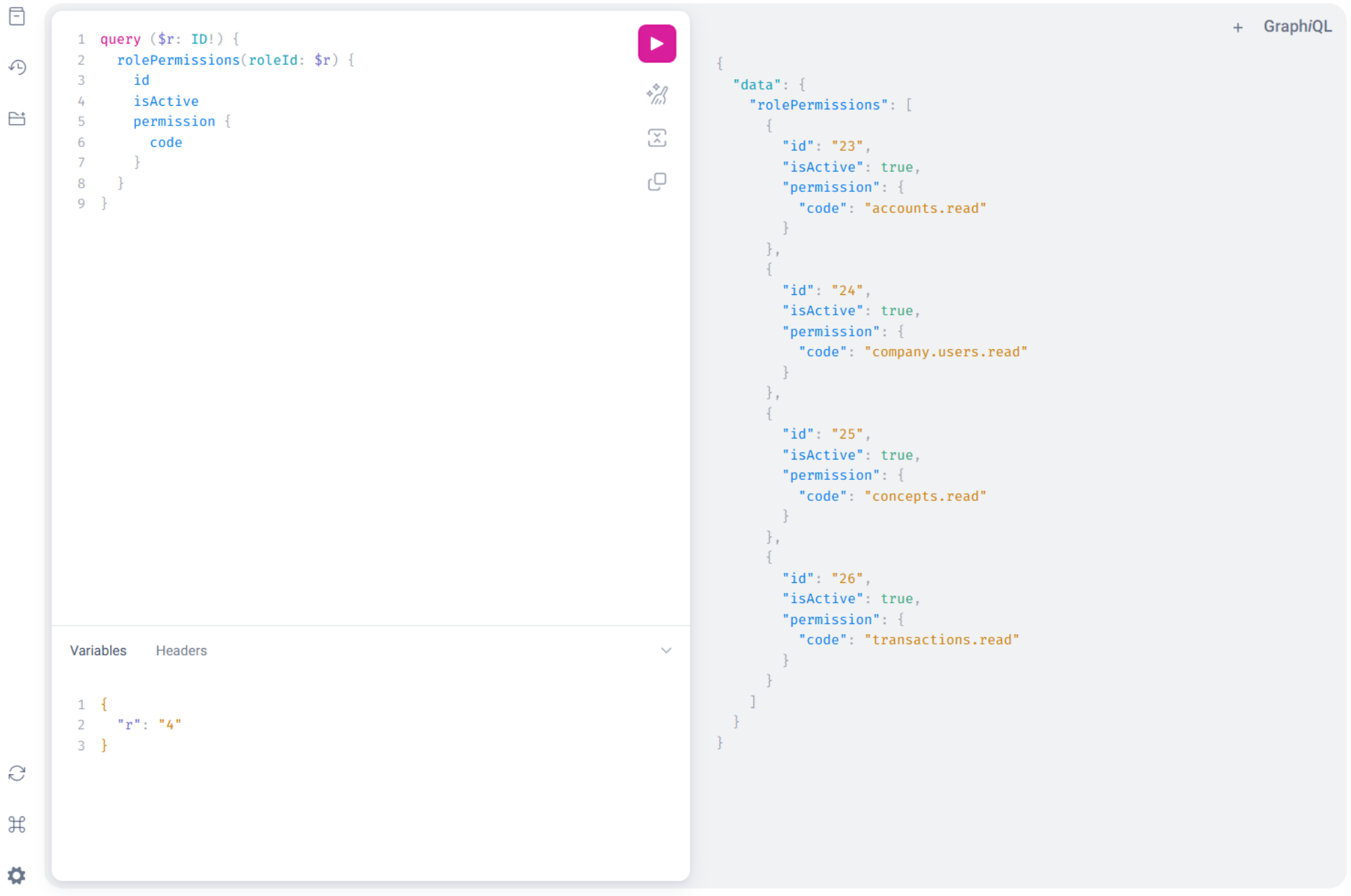
Resultado: OK.



18 Matriz final CONSULTA

Actor: Ana. Lista vigente del rol CONSULTA.

Resultado: OK.



La matriz activa vuelve a la configuración inicial

Resultado real de MySQL para la empresa de demostración. 1 = pareja activa; 0 = pareja conservada y desactivada. Los ceros documentan el historial sin borrar filas.

```
SELECT r.code AS rol, p.code AS permiso, rp.is_active
FROM role_permissions rp
JOIN roles r ON r.id = rp.role_id
JOIN permissions p ON p.id = rp.permission_id
WHERE r.company_id = :company_id
ORDER BY r.code, p.code
```

Rol	Permiso	is_active
ADMINISTRADOR	accounts.read	1
ADMINISTRADOR	accounts.write	1
ADMINISTRADOR	company.users.read	1
ADMINISTRADOR	company.users.write	1
ADMINISTRADOR	concepts.read	1
ADMINISTRADOR	concepts.write	1
ADMINISTRADOR	roles.read	1
ADMINISTRADOR	roles.write	1
ADMINISTRADOR	transactions.read	1
ADMINISTRADOR	transactions.write	1
CAPTURISTA	accounts.read	1
CAPTURISTA	accounts.write	0
CAPTURISTA	concepts.read	1
CAPTURISTA	transactions.read	1
CAPTURISTA	transactions.write	1
CONSULTA	accounts.read	1
CONSULTA	company.users.read	1
CONSULTA	concepts.read	1
CONSULTA	concepts.write	0
CONSULTA	transactions.read	1
CONTADOR	accounts.read	1
CONTADOR	accounts.write	1
CONTADOR	company.users.read	1
CONTADOR	concepts.read	1
CONTADOR	concepts.write	1
CONTADOR	roles.read	1
CONTADOR	transactions.read	1
CONTADOR	transactions.write	1

El reinicio conserva la configuración

Se reinició realmente el contenedor de la API mediante Docker Compose y se consultó MySQL de nuevo. La configuración mantuvo cuatro roles y diez permisos. También se comprobó la existencia del índice único del administrador principal.


```
{
  "database": "mysql",
  "server": "8.4.11",
  "roles": 4,
  "permissions": 10,
  "company_user_roles": 7,
  "admin_unique_index": true,
  "status": "PASS"
}
```


Las pruebas adicionales cubren permisos de lectura y escritura por separado, actor falsificado, aislamiento entre empresas, configuración revocada después de reiniciar, asignaciones concurrentes y restricciones de integridad incluso fuera de GraphQL.

Comandos para reproducir desde codigo:

```
python -m pytest -v
python scripts/test_mysql.py
python scripts/demo_roles.py --docker
```

Aviso de dependencias: Starlette/AnyIO emitió una deprecación de BlockingPortal. No hubo fallos de pruebas y pip check no encontró dependencias rotas.